

Computing Nonoverlapping Inversion Distance Between Two Strings in Linear Average Time

XIAODONG WANG¹ and LEI WANG²

ABSTRACT

Biological events like inversions are not automatically detected by the usual alignment algorithms. Alignment with inversions does not have a known polynomial time algorithm and Schöniger and Waterman introduced a simplification of the alignment problem with nonoverlapping inversions, where all regions will not be allowed to overlap. They presented an $\mathcal{O}(n^6)$ algorithm to compute nonoverlapping inversion distance between two strings of length n . The time and space complexities were improved to $\mathcal{O}(n^3)$ and $\mathcal{O}(n^2)$ later by Cho, Vellozo, and Ta. In this article, a linear space and linear average time algorithm to compute the inversion distance between two strings of the same length is presented. The recursive formula for this purpose is new to the best of our knowledge. The space costs of the algorithms to solve the same problem are quadratic in the literature, and thus our original algorithm is the first linear space and linear average time algorithm to solve the inversion distance problem.

Keywords: algorithms, combinatorics, optimization.

1. INTRODUCTION

COMPUTER ALIGNMENT OF MOLECULAR SEQUENCES is widely used for biological sequence comparisons to retrieve relevant information (Gao et al., 2003; Li and Ng, 2009; Sakai, 2011; Cantone et al., 2013; Huang et al., 2014). Inversions have recently been found to be polymorphic in the human genome (Feuk et al., 2005) and become very prominent in comparative genomics (Caceres et al., 1999; Cerdeño-Tárraga et al., 2005). The alignment with inversions problem for an unlimited alphabet size is NP-hard (non-deterministic polynomial-time hard) (Chen and Jiang, 2005). Some simplifications of this problem have been studied and were proved to be NP-complete (Wagner, 1975).

Schöniger and Waterman (1992) introduced a simplification of the alignment problem with nonoverlapping inversions, where all regions will not be allowed to overlap. For local DNA comparisons on relatively close sequences, such a simplification is realistic (Schöniger and Waterman, 1992). The core of the problem is to find the nonoverlapping inversion distance between two strings of the same length, that is, the alignment problem with nonoverlapping inversions. Schöniger and Waterman (1992) presented an $\mathcal{O}(n^6)$ algorithm to compute nonoverlapping inversion distance between two strings of length n . The time and space complexities were improved to $\mathcal{O}(n^4)$ and $\mathcal{O}(n^2)$, respectively, later by do Lago and Kulikowski (2005). An algorithm with $\mathcal{O}(n^3 \log n)$ time (Alves et al., 2005) was announced shortly. Finally, the time and space complexities were reduced to $\mathcal{O}(n^3)$ and $\mathcal{O}(n^2)$ later by Cho et al. (2015), Vellozo et al. (2006), and Ta et al. (2016).

¹Computer Science Department, Fujian University of Technology, Fuzhou, China.

²Facebook, Inc., Menlo Park, California.

In the nonoverlapping inversion distance problem, inversion occurs only in one string and transforms the string to the other string. A somewhat different problem, called the nonoverlapping inversion common ancestor problem, allows the nonoverlapping inversion operations to be applied on both strings simultaneously. The problem is more general than the nonoverlapping inversion distance problem. Cho et al. (2015) have presented an $\mathcal{O}(n^3)$ time algorithm to solve the nonoverlapping inversion common ancestor problem, using $\mathcal{O}(n^2)$ space, where n is the size of the two input strings. Very recently, Zohora and Rahman (2016) found through experimentation that the algorithm by Cho et al. (2015) fails in returning the correct answers in some cases because of not tracking the prefixes of the common ancestors. Founded on the idea of Cho et al. (2015), they finally found a correct algorithm to solve the nonoverlapping inversion common ancestor problem. Their algorithm requires $\mathcal{O}(n^4)$ time and $\mathcal{O}(n^3)$ space in the worst case. But, based on the experimental results reported in their article, they claimed that the practical running time of the algorithm in average and the worst case is $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$, respectively.

In this article, we are also interested in the computation of the inversion distance between two strings of the same length. A linear space and linear average time algorithm to compute the inversion distance between two strings of the same length is presented in this article. The recursive formula for this purpose is novel to the best of our knowledge. The space costs of the algorithms to solve the same problem are quadratic in the literature, and thus our original algorithm is the first linear space and linear average time algorithm to solve the inversion distance problem.

The organization of the article is as follows.

In the following four sections, we describe our algorithm for computing the nonoverlapping inversion distance between two strings of the same length.

In Section 2, the preliminary knowledge for presenting our algorithm is discussed. In Section 3, we present a very simple dynamic programming algorithm to compute the inversion distance of two input strings of the same length n . In Section 4, the time and space complexities of the algorithm are analyzed. Our algorithm can compute the nonoverlapping inversion distance between two strings of the same length n in average $\mathcal{O}(n)$ time, using $\mathcal{O}(n)$ space. Some concluding remarks are placed in Section 5.

2. PRELIMINARIES

In the whole article, we use $x = x_1x_2 \cdots x_n$ and $y = y_1y_2 \cdots y_n$ to denote the two input strings of size n over a finite alphabet Σ .

Some concepts on the inversion distance problem are fixed in the following way.

For a given string x of length n , the i th character of x is denoted as $x_i \in \Sigma$ for any $i = 1, \dots, n$. A substring of x from position i to j can be denoted as $x_{i,j} = x_ix_{i+1} \cdots x_j$, $1 \leq i, j \leq n$.

If $i \neq 1$ or $j \neq n$, then the substring $x_{i,j}$ is called a proper substring of x . A substring $x_{i,j}$ is called a prefix or a suffix of x if $i = 1$ or $j = n$, respectively. For any prefix $x_{1,j}$ of x , the substring $x_{k,j}$, $1 \leq k \leq j$, is also called the suffix of $x_{1,j}$ starting at the position k . For the two prefixes $x_{1,i}$ of x and $y_{1,j}$ of y , if a substring s is a suffix of both $x_{1,i}$ and $y_{1,j}$, then s is called a common suffix of $x_{1,i}$ and $y_{1,j}$.

In biological sequences, $\Sigma = \{A, C, G, T\}$, in which $A - T$ and $C - G$ are considered as complementary base pairs.

Definition 1.

For each character $c \in \Sigma$, $\theta(c)$ is defined as its complementary character in Σ .

In particular, $\theta(A) = T$, $\theta(T) = A$, $\theta(G) = C$, and $\theta(C) = G$.

For any substring $x_{i,j}$ of x , the inversion operation θ acting on $x_{i,j}$ is defined as

$$\theta(x_{i,j}) = \theta(x_j)\theta(x_{j-1}) \cdots \theta(x_i) \quad (1)$$

$\theta(x_{i,j})$ is also denoted as $\theta_{i,j}(x)$.

Definition 2.

For the inversion operation $\theta_{i,j}$, the index range $[i, j]$ is called its inversion range, and for $t \in [i, j]$, the operations $\theta_{i,j}$ cover t . For any two inversion operations, if the intersection of their inversion ranges is empty, then they are called nonoverlapping inversion operations.

Definition 3.

For given two strings x and y of the same length, the minimum number of nonoverlapping inversion operations used to transform x into y , denoted by $id(x, y)$, is called the inversion distance between x and y . If there is no set of nonoverlapping inversion operations can convert x into y , then $id(x, y)$ is infinite.

Definition 4.

If a set Θ of nonoverlapping inversion operations consecutively applied on the string x can convert x into y , then the set Θ is called a feasible inversion operation set, and is denoted as $\Theta(x)=y$. It is clear that if $\theta_{j,i} \in \Theta$, then $\theta_{j,i}(x)=y_{j,i}$. The inversion operations in a feasible inversion operation set Θ are called feasible inversion operations. For each $1 \leq i \leq n$, the feasible inversion operation set up to i is denoted as

$$\xi(i) = \{j | 1 \leq j \leq i, \theta_{j,i}(x) = y_{j,i}\}. \quad (2)$$

The number of inversion operations in Θ is denoted by $|\Theta|$. In the case of $|\Theta| = id(x, y)$, the set Θ is called an optimal inversion operation set of x and y . $ID(x, y)$ is used to denote the set of optimal inversion operation set of x and y .

$$ID(x, y) = \{\Theta | \Theta \text{ is an optimal inversion operation set of } x \text{ and } y\} \quad (3)$$

The set $\xi(i)$ can be found recursively as follows.

Theorem 1.

For each pair (j, i) , $1 \leq j \leq i \leq n$, $j \in \xi(i)$ if and only if

- $\theta(x_i) = y_j$ and $\theta(x_j) = y_i$.
- $j+1 \in \xi(i-1)$ if $i-j \geq 2$.

Proof.

(1) In the case of $i-j \geq 2$, we have $\theta_{j,i}(x) = \theta(x_i)\theta_{j+1,i-1}(x)\theta(x_j)$.

It follows that $j \in \xi(i)$ if and only if $\theta(x_i)\theta_{j+1,i-1}(x)\theta(x_j) = y_{j,i}$. It is equivalent to

$$\begin{cases} \theta(x_i) = y_j \\ \theta_{j+1,i-1}(x) = y_{j+1,i-1} \\ \theta(x_j) = y_i \end{cases}$$

Thus,

$$\begin{cases} \theta(x_i) = y_j \\ j+1 \in \xi(i-1) \\ \theta(x_j) = y_i \end{cases}$$

(2) The lemma is trivially true in the case of $i-j < 2$.

The proof is completed. $\square$

In the following discussion, the inversion distance between $x_{1,i}$ and $y_{1,i}$ is denoted by $f(i) = id(x_{1,i}, y_{1,i})$, $0 \leq i \leq n$. The nonoverlapping inversion distance problem is to find $f(n) = id(x, y)$.

3. THE DESCRIPTION OF THE ALGORITHM

In this section, we discuss how to find $f(i)$, $1 \leq i \leq n$, iteratively. There are two cases to be distinguished.

In the first case, let $f(i-1) < \infty$ and $x_i = y_i$. It is clear that there must be a set Θ_{i-1} of nonoverlapping inversion operations such that $f(i-1) = |\Theta_{i-1}|$, and Θ_{i-1} can convert $x_{1,i-1}$ into $y_{1,i-1}$. It follows from $x_i = y_i$ that Θ_{i-1} can also convert $x_{1,i}$ into $y_{1,i}$. It follows that in this case, $f(i) \leq f(i-1)$.

In the second case, let $j \in \xi(i)$ and $f(j-1) < \infty$. In this case, there must be a set Θ_{j-1} of nonoverlapping inversion operations such that $f(j-1) = |\Theta_{j-1}|$, and Θ_{j-1} can convert $x_{1,j-1}$ into $y_{1,j-1}$. It follows from $j \in \xi(i)$ that $\Theta_{j-1} \cup \{\theta_{j,i}\}$ can convert $x_{1,i}$ into $y_{1,i}$, and $|\Theta_{j-1} \cup \{\theta_{j,i}\}| = f(j-1) + 1 < \infty$. It is clear that $\Theta_{j-1} \cup \{\theta_{j,i}\}$ must be a candidate of Θ_i , and thus $f(i) \leq f(j-1) + 1$.

The two values $\alpha(i)$ and $\beta(i)$ corresponding to the two cases already discussed can be defined as follows.

$$\alpha(i) = \begin{cases} f(i-1) & \text{if } x_i = y_i \\ \infty & \text{Otherwise} \end{cases} \quad (4)$$

$$\beta(i) = \begin{cases} \min_{j \in \xi(i)} f(j-1) + 1 & \text{if } \xi(i) \neq \emptyset \\ \infty & \text{Otherwise} \end{cases} \quad (5)$$

In the following theorem, we can show a new recursive formula to compute $f(i)$ iteratively for all $1 \leq i \leq n$.

Theorem 2.

For each $1 \leq i \leq n$,

$$f(i) = \min \begin{cases} \alpha(i) \\ \beta(i) \end{cases} \quad (6)$$

Proof.

The theorem can be proved by induction on n the size of the input string x . The theorem is trivially true for the cases of $n \leq 1$. Suppose the theorem is true when the size of the input string x is $< n$. We now show that when the size of the input string x is n , the theorem is also true. ■

Let

$$f(n) = id(x, y), \quad l = \min \begin{cases} \alpha(n) \\ \beta(n) \end{cases}$$

There are two cases to be distinguished.

(1) $f(n) < \infty$.

In this case, x can be converted into y by a set of nonoverlapping inversion operations. For any feasible inversion operation set $\Theta = \{O_1, \dots, O_p\}$, we can prove $|\Theta| = p \geq l$.

Let the inversion range of O_t be $[j_t, i_t]$, $1 \leq t \leq p$. Since the inversion operations in Θ are mutually nonoverlapping, we can assume $1 < i_1 < i_2 < \dots < i_p$. It is clear that if the t inversion operations $O_1, \dots, O_t$ are consecutively applied on x , then x_{1, i_t} can be converted to y_{1, i_t} successively.

There are also two subcases to be distinguished.

(1.1) $i_p < n$.

It is clear that in this case, $x_n = y_n$. It follows that $g(n-1) = p \geq f(n-1) = \alpha(n)$. It follows from $\alpha(n) \geq l$ that $p \geq l$.

(1.2) $i_p = n$.

In this case, $O_p = \theta_{j_p, n}$, and thus $j_p \in \xi(n)$. It follows from Equation (5) that $\beta(n) \leq f(j_p-1) + 1$. In contrast, it follows from $i_{p-1} \leq j_p-1 < j_p$ and $f(j_p-1) < \infty$ that $f(j_p-1) = p-1 \geq \beta(n)-1$. It follows that $p \geq \beta(n) \geq l$. Especially, for any $\Theta \in ID(x, y)$, $|\Theta| \geq l$. It follows that

$$f(n) \geq \min \begin{cases} \alpha(n) \\ \beta(n) \end{cases} \quad (7)$$

In contrast, if $l = \alpha(n)$, then $x_n = y_n$. Let $\Theta \in ID(x_{1, n-1}, y_{1, n-1})$. It is clear that $\Theta(x) = y$. It follows that $f(n) \leq |\Theta| = f(n-1) = l$.

In the case of $l = \beta(n) = f(j-1) + 1$, $1 \leq j < n$, let $\Theta' \in ID(x_{1, j-1}, y_{1, j-1})$, and $\Theta = \Theta' \cup \{\theta_{j, n}\}$. It is clear that $\Theta(x) = y$. It follows that $f(n) \leq |\Theta| = f(j-1) + 1 = \beta(n) = l$. Finally, we have

$$f(n) \leq \min \begin{cases} \alpha(n) \\ \beta(n) \end{cases} \quad (8)$$

It follows from Equations (7) and (8) that

$$f(n) = \min \begin{cases} \alpha(n) \\ \beta(n) \end{cases} \quad (9)$$

(2) $f(n) = \infty$.

In this case, x cannot be converted into y by any set of nonoverlapping inversion operations. It is readily seen that in this case, $\alpha(n) = \infty$ and $\beta(n) = \infty$. It follows that

$$f(n) = \infty = \min \begin{cases} \alpha(n) \\ \beta(n) \end{cases} \quad (10)$$

The proof is completed. $\square$

Based on Theorem 1, the inversion distance between the given input strings $x = x_1x_2 \cdots x_n$ and $y = y_1y_2 \cdots y_n$ of the same length n can be computed by a simple dynamic programming algorithm as follows.

Algorithm 1: $id(x, y)$

Input: Two strings x, y of the same length n .

Output: The inversion distance $id(x, y)$.

```

1 for  $i = 1$  to  $n$  do
2    $f(i) \leftarrow \infty$ ;
3   if  $x_i = y_i$  then  $f(i) \leftarrow f(i-1)$ ;
4    $f(i) \leftarrow \min \{f(i), \beta(i)\}$ ;
5 end
```

The key computation of Algorithm 1 is to compute $\beta(i)$ for $1 \leq i \leq n$. It is clear that only inversion operations $\theta_{j,i}$ such that $j \in \xi(i)$ are feasible operations, and thus only these values of $f(j-1) + 1$ need to be compared with the value of $f(i)$. The sets $\xi(i)$, $1 \leq i \leq n$ can be computed iteratively according to Lemma 1. Therefore, the values of $\beta(i)$, $1 \leq i \leq n$ can be computed iteratively as follows.

Algorithm 2: $\beta(i)$

Input: i , $1 \leq i \leq n$.

Output: $\beta(i)$.

```

1 while  $Q \neq \emptyset$  do
2    $j \leftarrow Q.front()$ ;
3    $Q.pop()$ ;
4   if  $j = 0$  then Break;
5   else if  $j > 1$  then check( $j, i$ );
6 end
7 for  $k = 0$  to  $1$  do check( $i + k, i$ );
8  $Q.push(0)$ ;
```

In Algorithm 2, we use a queue Q to store the set $\xi(i-1)$ and $\xi(i)$ simultaneously. The value 0 is utilized to separate the two sets. The elements of $\xi(i-1)$ are in the first part of Q , and the elements of $\xi(i)$ are in the following part. For each $j \in \xi(i-1)$, the inversion operation $\theta_{j-1,i}$ is checked for its feasibility by Lemma 1. If it is a feasible inversion operation, then $j-1$ is pushed into queue Q , and the value $f(j-2) + 1$ is

compared with $f(i)$. The special cases of $j=i$ and $j=i-1$ are checked separately. The task $check(j, i)$ can be described as follows.

Algorithm 3: $check(j, i)$

Input: $j, i, 1 \leq j \leq i \leq n$.
1 if $\theta(x_{j-1}) = y_i$ and $\theta(x_i) = y_{i-1}$ then
2 $Q.push(j-1)$;
3 if $f(j-2) + 1 < f(i)$ then
4 $f(i) \leftarrow f(j-2) + 1$;
5 $t(i) \leftarrow j-1$;
6 end
7 end

In Algorithm 3, an array t is used to store the information on the feasible inversion operations found by the algorithm, which can be used to rebuild an optimal inversion operation set in $ID(x, y)$ at the end of Algorithm 1. With the information stored in the array t , an optimal inversion operation set in $ID(x, y)$ can be constructed by a recursive algorithm as follows.

Algorithm 4: $Backtrack(i)$

1 if $i < 1$ then return;
2 if $x_i = y_i$ and $f(i) = f(i-1)$ then $Backtrack(i-1)$;
3 else
4 $Backtrack(t(i) - 1)$;
5 print $(t(i), i)$;
6 end
7 return;

A call $Backtrack(n)$ will produce an optimal inversion operation set in $ID(x, y)$ in $\mathcal{O}(n)$ time.

Example.

Let $\Sigma = \{A, C, G, T\}$, $x = TAGGAC$, and $y = TCTGAG$.

The sets $\xi(i)$, $1 \leq i \leq 6$ found by the algorithm are $\xi(3) = \{2\}$ and $\xi(6) = \{6\}$.

The inversion distance $f[i]$ can then be computed by Algorithm 1.

$$f = (0, \infty, 1, 1, 1, 2).$$

The array t to store the information on the feasible inversion operations is also computed.

$$t = (0, 0, 2, 0, 0, 6).$$

Therefore, $f(n) = id(x, y) = 2$. Finally, a call $Backtrack(n)$ produces an optimal inversion operation set $\{\theta_{2,3}, \theta_{6,6}\} \in ID(x, y)$.

4. THE TIME AND SPACE COMPLEXITIES OF THE ALGORITHM

In Algorithm 1, $\alpha(i)$ is computed in line 3, and $\beta(i)$ is computed in line 4, and then $f(i)$ is computed by Equation (6) in line 4. The correctness of Algorithm 1 can be verified according to Theorem 2 and the discussion is mentioned previously.

Two arrays f and t of size n and a queue Q are used in the algorithm. At any point of the algorithm, the queue Q contains at most two sets $\xi(i-1)$ and $\xi(i)$ for some i , and thus $\mathcal{O}(n)$ space suffices for Q . Therefore, the space complexity of the algorithm is clearly $\mathcal{O}(n)$.

It is clear that $|\xi(i)| \leq i$ and thus $\beta(i)$ may cost $\mathcal{O}(i)$ time in the worst case. It follows that the time complexity of Algorithm 1 would be $\mathcal{O}(n^2)$ in the worst case. This is the best result so far compared with the $\mathcal{O}(n^3)$ worst-case time results reported previously in Cho et al. (2015), Vellozo et al. (2006), and Ta et al. (2016). Furthermore, we can prove that the size of $\xi(i)$ is much smaller than i on average, and thus $\beta(i)$ can cost $\mathcal{O}(1)$ time in the average case.

In the following discussion, assume $\Sigma = \{c_1, c_2, \dots, c_\sigma\}$ to be a finite alphabet, where $\sigma \geq 4$ is a constant and assumes a uniform distribution and independence of characters.

The average time cost to compute $\beta(i)$ is denoted as $t_\beta(i)$, for each $1 \leq i \leq n$. Then, the total average time in computing $\beta(i)$, $1 \leq i \leq n$, must be

$$T(n) = \sum_{i=1}^n t_\beta(i). \quad (11)$$

It follows from Algorithm 2 that it costs $\mathcal{O}(1)$ time for each $j \in \xi(i)$ in the computation of $\beta(i)$. It follows that

$$T(n) = \sum_{1 \leq j \leq i \leq n} \Pr\{j \in \xi(i)\} \mathcal{O}(1) = \mathcal{O}\left(\sum_{1 \leq j \leq i \leq n} \Pr\{j \in \xi(i)\}\right).$$

If the inversion string of x is denoted as

$$z = \theta(x) = \theta(x_n)\theta(x_{n-1}) \cdots \theta(x_1), \quad (12)$$

then for each pair (j, i) , $\theta_{j,i}(x) = z_{n-i+1, n-j+1}$. Let

$$\eta(i) = \{j \mid 1 \leq j \leq i, z_{n-i+1, n-j+1} \text{ is a permutation of } y_{j,i}\}. \quad (13)$$

Then, by the definition of $\xi(i)$, for each pair (j, i) , if $j \in \xi(i)$, then $\theta_{j,i}(x) = y_{j,i}$, and thus $j \in \eta(i)$. It follows that $\xi(i) \subseteq \eta(i)$, for each $1 \leq i \leq n$. Therefore, $T(n)$ can be bounded above by

$$\mathcal{O}\left(\sum_{1 \leq j \leq i \leq n} \Pr\{j \in \eta(i)\}\right).$$

For each pair (j, i) such that $j \in \eta(i)$ denote the number of occurrences of $c_t \in \Sigma = \{c_1, \dots, c_\sigma\}$ in $y_{j,i}$ and $z_{n-i+1, n-j+1}$ as $g_y(j, i, t)$ and $g_z(j, i, t)$, respectively. It is clear that $z_{n-i+1, n-j+1}$ is a permutation $y_{j,i}$ if and only if $g_z(j, i, t) = g_y(j, i, t)$, $1 \leq t \leq \sigma$. For simplicity, denote $m = i - j + 1$ and $s_t = g_y(j, i, t)$ for fixed (j, i) , $1 \leq j \leq i \leq n$, and $1 \leq t \leq \sigma$. It follows that the probability that $z_{n-i+1, n-j+1}$ is a permutation $y_{j,i}$ is exactly

$$\Pr\{j \in \eta(i)\} \leq \frac{\binom{m}{s_1} \binom{m-s_1}{s_2} \binom{m-s_1-s_2}{s_3} \cdots \binom{s_\sigma}{s_\sigma}}{\sigma^m}. \quad (14)$$

Let $k = \lfloor m/\sigma \rfloor$ and, without loss of generality, we assume that σ divides m . It is clear that the probability given in Equation (14) is maximized when $s_t = k$ for all $1 \leq t \leq \sigma$. It follows that

$$\Pr\{j \in \eta(i)\} \leq \frac{\binom{m}{k} \binom{m-k}{k} \binom{m-2k}{k} \cdots \binom{k}{k}}{\sigma^m} = \frac{m!}{(k!)^\sigma \sigma^m}.$$

It follows from Stirling's approximation for both $m!$ and $k!$ that

$$\frac{m!}{(k!)^\sigma \sigma^m} = \Theta\left(\frac{\sqrt{2\pi m}(m/e)^m}{(\sqrt{2\pi(m/\sigma)}(m/(e\sigma))^{m/\sigma})^\sigma \sigma^m}\right) = \Theta\left(\frac{\sqrt{2\pi m}}{(\sqrt{2\pi(m/\sigma)})^\sigma}\right).$$

It is clear that $\sqrt{2\pi}/(\sqrt{2\pi})^\sigma \leq 1$, and thus

$$\Theta\left(\frac{\sqrt{m}}{\left(\sqrt{m/\sigma}\right)^\sigma}\right) = \Theta\left(\frac{\sigma^{\sigma/2}}{m^{(\sigma-1)/2}}\right).$$

It follows that

$$\Pr\{j \in \eta(i)\} \leq \frac{\sigma^{\sigma/2}}{(i-j+1)^{(\sigma-1)/2}}.$$

Therefore,

$$T(n) = \mathcal{O}\left(\sum_{1 \leq j \leq i \leq n} \Pr\{j \in \eta(i)\}\right) = \mathcal{O}\left(\sum_{1 \leq j \leq i \leq n} \frac{\sigma^{\sigma/2}}{(i-j+1)^{(\sigma-1)/2}}\right).$$

It follows from the assumption that $\sigma \geq 4$ is a constant that $\frac{\sigma^{\sigma/2}}{(i-j+1)^{(\sigma-1)/2}} = \mathcal{O}\left(\frac{1}{(i-j+1)^{1.5}}\right)$. It is well known that $\sum_{i=1}^{\infty} \frac{1}{i^{1.5}} = 2.61238 \dots$, and thus

$$T(n) = \mathcal{O}\left(\sum_{1 \leq j \leq i \leq n} \frac{1}{(i-j+1)^{1.5}}\right) = \mathcal{O}(n). \quad (15)$$

Theorem 3.

Algorithm 1 computes the inversion distance of two equal length strings in $\mathcal{O}(n)$ average time and $\mathcal{O}(n)$ space.

Proof.

It has been shown by Equation (15) that $T(n)$, the average time cost to compute $\beta(i)$, $1 \leq i \leq n$, can be bounded above by $\mathcal{O}(n)$. The time cost of the other parts of Algorithm 1 is clearly $\mathcal{O}(n)$. Therefore, the time cost of Algorithm 1 must be $\mathcal{O}(n)$ in average. The space cost of Algorithm 1 is clearly $\mathcal{O}(n)$.

The proof is completed. $\square$

5. CONCLUDING REMARKS

We have presented a linear average time and linear space algorithm to compute the inversion distance between two strings of the same length. The recursive formula for this purpose is very simple. The algorithm cannot be applied directly to the nonoverlapping inversion common ancestor problem. The best upper bound of the time cost to solve this problem in the literature is $\mathcal{O}(n^4)$ in the worst case.

It is not clear to us how our algorithm to solve the inversion distance problem can be modified to solve the nonoverlapping inversion common ancestor problem. This leaves room for further exploration.

ACKNOWLEDGMENTS

This work was supported by Fujian Provincial Key Laboratory of Data-Intensive Computing and Fujian University Laboratory of Intelligent Computing and Information Processing.

AUTHOR DISCLOSURE STATEMENT

The authors declare that no competing financial interests exist.

REFERENCES

- Alves, C.E.R., do Lago, A.P., Vellozo, A.F. 2005. Alignment with non-overlapping inversions in $o(n^3 \log n)$ time. *Electron. Notes Discrete Math.* 19, 365–371.
- Caceres, M., Ranz, J.M., Barbadilla, A., et al. 1999. Generation of a widespread drosophila inversion by a transposable element. *Science*. 285, 415–418.
- Cantone, D., Cristofaro, S., and Faro, S. 2013. Efficient string-matching allowing for non-overlapping inversions. *Theoret. Comput. Sci.* 483, 85–95.
- Cerdeño-Tárraga, A.M., Patrick, S., Crossman, L.C., et al. 2005. Extensive DNA inversions in the *B. fragilis* genome control variable gene expression. *Science*. 307, 1463–1465.
- Chen, X., and Jiang, T. 2005. Assignment of orthologous genes via genome rearrangement. *IEEE ACM Trans. Comput. Biol. Bioinform.* 2, 302–315.
- Cho, D., Han, Y., and Kim, H. 2015. Alignment with non-overlapping inversions and translocations on two strings. *Theoret. Comput. Sci.* 575, 90–101.
- do Lago, A.P., and Kulikowski, C. 2005. A sparse dynamic programming algorithm for alignment with non-overlapping inversions. *Theor. Inform. Appl.* 39, 175–189.
- Feuk, L., MacDonald, J.R., Tang, T., et al. 2005. Discovery of human inversion polymorphisms by comparative analysis of human and chimpanzee DNA sequence assemblies. *PLoS Genet.* 1, e56.
- Gao, Y., Wu, J., and Niewiadomski, R. 2003. A space efficient algorithm for sequence alignment with inversions. In *Proceedings of the COCOON 2003*, volume 2697 of *Lecture Notes in Computer Science*, 57–67. Springer-Verlag, London.
- Huang, S., Yang, C., and Ta, T. 2014. Approximate string matching problem under non-overlapping inversion distance. In *Proceedings of the 2014 International Computer Symposium*, volume 1 of *ICS'14*, 38–46. IOS Press, Amsterdam, The Netherlands.
- Li, S., and Ng, Y. 2009. On protein structure alignment under distance constraint. In *Proceedings of the ISAAC 2009*, volume 5878 of *Lecture Notes in Computer Science*, 65–76. Springer-Verlag, London.
- Sakai, Y. 2011. A new algorithm for the characteristic string problem under loose similarity criteria. In *Proceedings of the ISAAC 2011*, volume 7074 of *Lecture Notes in Computer Science*, 663–672. Springer-Verlag, London.
- Schöniger, M., and Waterman, M. 1992. A local algorithm for DNA sequence alignment with inversions. *Bull. Math. Biol.* 54, 521–536.
- Ta, T., Lin, C., and Lu, C. 2016. An efficient algorithm for computing non-overlapping inversion and transposition distance. *Inform. Process. Lett.* 116, 744–749.
- Vellozo, A., Alves, C., and do Lago, A. 2006. Alignment with non-overlapping inversions in $o(n^3)$ time. In *Proceedings of the WABI 2006*, volume 4175 of *Lecture Notes in Computer Science*, 186–196. Springer-Verlag, London.
- Wagner, R. 1975. On the complexity of the extended string-to-string correction problem. In *Proceedings of the 7th Annual ACM Symposium on Theory of Computing*, volume 1 of *STOC'75*, 218–223. IOS Press, Amsterdam, The Netherlands.
- Zohora, F.T., and Rahman, M.S. 2016. An efficient algorithm to detect common ancestor genes for non-overlapping inversion and applications. *Theoret. Comput. Sci.* 656, 188–214.

Address correspondence to:

Prof. Xiaodong Wang
Computer Science Department
Fujian University of Technology
Fuzhou 350118
China

E-mail: wangxd135@139.com

Dr. Lei Wang
Facebook, Inc.
Menlo Park, CA 94052

E-mail: soft139@qq.com